

Technical Architecture Document

DartsGame — Godot 4 Darts Simulation

Prepared by: Claude Code | Date: 29 June 2026 | Version: 1.0

1. Project Overview

DartsGame is a single-player darts simulation built in Godot 4, targeting a 1920×1080 broadcast-style presentation. The player competes against a scripted AI opponent in standard darts formats (301/501, legs or sets, straight-in or double-in, double-out always). All rendering is procedural — no sprites or texture assets are used for gameplay elements. The visual layout mimics a broadcast split-screen: the left ~900px hosts the scoreboard and stats panel; the right ~1010px centres the dartboard at pixel coordinates (1410, 540).

2. Technology Stack

Concern	Choice
Engine	Godot 4 (GDScript)
Resolution	1920×1080 fixed
Rendering	Godot immediate-mode _draw API (Canvas2D)
UI	Godot Control nodes with manual layout (no theme system)
Asset dependencies	None — font via ThemeDB.fallback_font only

3. Scene Graph

The main scene is a flat composition of named nodes. Each functional node carries a GDScript. Static UI chrome (dividers, tint rects) lives under BoardCameraZone with no script.

```
Main (Node2D)
  ■■■ BroadcastBackdrop (Node2D) broadcast_backdrop.gd
  ■■■ BoardCameraZone (Control) — layout chrome only
    ■ ■■■ ScoreboardFalloff (ColorRect)
    ■ ■■■ DividerGlow (ColorRect)
    ■ ■■■ DividerLine (ColorRect)
    ■ ■■■ BoardCameraTint (ColorRect)
  ■■■ ScoreboardLayer (Control) scoreboard_presenter.gd
  ■ ■■■ CheckoutRoute (Control)
  ■ ■ ■■■ TopRouteTile1..3
  ■ ■ ■■■ RouteTile1..3
  ■ ■■■ MatchScoreboard (Control)
  ■ ■ ■■■ PlayerRow
  ■ ■ ■■■ AIRow
```

- ■■■ StatsPanel (Control)
- ■■■ CurrentLeg
- ■■■ MatchStats
- Dartboard (Node2D) dartboard.gd
- ThrowLayer (Node2D) throw_controller.gd
- MatchSetupModal (Control) match_setup_modal.gd
- ■■■ Panel (ColorRect)
- ■■■ ScoreOption / FormatOption / BestOfOption / StarterOption
- ■■■ DoubleInToggle
- ■■■ StartButton
- StatsSummaryModal (Control) stats_summary_modal.gd
- Panel (ColorRect)
- SummaryTable (GridContainer, 3 cols)
- BreakdownScroll → BreakdownTable (GridContainer, 6 cols)

4. Script Responsibilities

4.1 throw_controller.gd — Game Logic Authority

This is the sole source of truth for all game state. Every other script either reads from it via NodePath label references, or receives instructions from it via method calls. It extends Node2D and handles both drawing (aim guides, dart markers) and all game logic.

4.1.1 Phase State Machine

```

THROW_PHASE_LOCK_HEIGHT (0) aim line sweeps vertically
↓ [click / Space]
THROW_PHASE_LOCK_WIDTH (1) aim line sweeps horizontally
↓ [click / Space]
THROW_PHASE LANDED (2) dart drawn; delay before next dart or AI turn
↓ [player_turn_end_delay]
THROW_PHASE_AI_THROWING (3) AI processes darts on timer
↓ [all darts thrown + ai_turn_end_delay]
■■■ loop back to LOCK_HEIGHT for next visit ■■■
THROW_PHASE_MATCH_COMPLETE (4) input disabled, match over

```

4.1.2 Aim Mechanics

The sweeping aim line uses `_triangle_sweep(time, speed, radius)`: a triangle wave (ping-pong) over `[-AIM_RADIUS, +AIM_RADIUS]`. A `_visible_wobble()` term adds two overlapping sine waves (3.2 Hz and 7.1 Hz) whose amplitude scales with `control_skill` (reduces wobble) and `pressure` (increases wobble). The horizontal sweep is constrained to `_safe_axis_limit(locked_y)` — the chord half-length at the locked y-coordinate — ensuring the crosshair cannot escape the circular aim area.

4.1.3 Scoring Pipeline (per dart)

```

_score_position(local_pos) → result dict
_apply_result_to_score(thrower, result)
■■■ double_in check → set is_not_in if not yet in
■■■ _is_bust(current_score, result)
next_score < 0 → bust
next_score == 1 → bust (unreachable finish)
next_score == 0 and
not double/bull → bust (must double-out)
■■■ [not bust] decrement score
_record_dart_progress(thrower, ...) → dart-level stat accumulation

```

4.1.4 Visit Finalization

`_finalize_visit()` is called once per visit, guarded by `_visit_finalized`. It records the visit score (0 if bust), updates checkout attempt/success counters, tracks high checkout, calls `_record_visit_thresholds` for 100+/140+/180 counting, and emits `visit_complete`. A visit that starts on a checkout-eligible score counts as a checkout attempt even if the player busts.

4.1.5 AI Targeting

```
_next_ai_target_label():
if double_in and not yet in → "D20"
elif checkout route available → route[0]
else → AI_DART_LABELS[sequence_index % 24]

AI_DART_LABELS is a 24-entry cycling script of realistic scoring targets:
T20, S20, S1, T19, S20, S5, T20, T20, S20 ...

Position computed in _local_position_for_target_label():
angle = sector_angle + _ai_angle_offset(dart_number)
radius = _ai_radius_for_multiplier(prefix, dart_number)
Both use sin() with (sequence_index + dart_number) seed for
consistent but varied grouping across the match.
```

4.1.6 Checkout Route Solver

_checkout_route_for_score(score, max_darts) uses a three-pass greedy search:

- Pass 1 (1 dart): direct finish — bull=50, even 2–40 as doubles
- Pass 2 (2 darts): iterate FINISH_DART_LABELS, then ROUTE_DART_LABELS for setup
- Pass 3 (3 darts): nested loop — ROUTE_DART_LABELS × ROUTE_DART_LABELS × FINISH_DART_LABELS

Bogey numbers (159, 162, 163, 165, 166, 168, 169) are explicitly rejected. max_darts limits search depth so mid-visit routes only show reachable finishes. The solver is $O(n^3)$ in the worst case; acceptable given array sizes (~21 finish labels, ~41 route labels) but results are not cached.

4.1.7 Match Progression

```
_reset_leg()
■■■ [first_thrower == AI] → _begin_ai_visit()
■■■ [first_thrower == PLAYER] → _begin_player_visit()

_complete_leg(winner)
■■■ MATCH_FORMAT_LEGS:
■ increment leg count
■ if legs >= target → _complete_match()
■ else → _reset_leg()
■■■ MATCH_FORMAT_SETS:
increment leg count for current set
if legs >= 2 → _complete_set(winner)
if sets >= target → _complete_match()
else → _show_set_complete_summary()
[user clicks Next Set]
continue_after_summary()
→ reset leg counts, _reset_leg()
else → _reset_leg()

Leg alternation: _next_leg_first_thrower toggles after every leg.
```

4.1.8 Signals

Signal	Payload	Consumer
dart_landed	result dict (label, score, multiplier, position, is_miss, is_bust, thrower)	External hookpoint; unused internally
visit_complete	results Array, counted_total int	External hookpoint; unused internally

4.1.9 Configurable Exports

Group	Export	Default	Effect
Match	starting_score	501	Initial score per leg
Match	match_format	Legs	Legs or Sets mode

Group	Export	Default	Effect
Match	best_of_count	5	Total legs/sets in match
Match	starting_thrower_selection	Player	Who throws first each match
Match	double_in_enabled	false	Must hit double to start scoring
Match	darts_per_visit	3	Darts per turn
Throw Feel	vertical_speed	430	Pixels/sec for height sweep
Throw Feel	horizontal_speed	500	Pixels/sec for width sweep
Throw Feel	visible_wobble_pixels	8	Base wobble amplitude
Throw Feel	control_skill	0.65	0–1; reduces wobble
Throw Feel	pressure	0.25	0–1; increases wobble & speed
Throw Feel	player_turn_end_delay	0.65 s	Pause after player visit ends
Throw Feel	ai_dart_interval	0.62 s	Time between AI darts
Throw Feel	ai_turn_end_delay	0.78 s	Pause after AI visit ends

4.2 dartboard.gd — Procedural Board Renderer

Marked @tool so it renders in the Godot editor. Extends Node2D, draws everything in `_draw()` using Godot's immediate-mode canvas API. No sprites or textures.

Draw call	Method	Notes
Shadow	<code>_draw()</code> inline	Offset circle behind board
Number ring	<code>_draw()</code> inline	Dark outer band
Numbered sections	<code>_draw_numbered_sections</code>	20 wedges × 4 ring segments each
Bull	<code>_draw_bull</code>	Outer green then inner red circles
Wires	<code>_draw_wires</code>	7 <code>draw_arc</code> + 20 radial <code>draw_line</code> calls
Numbers	<code>_draw_numbers</code>	Polar position, drop-shadow + fill pass

Each ring segment is built as a `PackedVector2Array`: outer-arc points forward, inner-arc points reversed, closed polygon. All geometry is parameterised by the 7 ring radii constants below.

```
INNER_BULL_RADIUS = 14 px OUTER_BULL_RADIUS = 35 px
TREBLE_INNER = 190 px TREBLE_OUTER = 222 px
DOUBLE_INNER = 318 px DOUBLE_OUTER = 350 px
NUMBER_RING_OUTER = 415 px
```

These constants are duplicated in `throw_controller.gd` for hit detection. See Section 7.

4.3 scoreboard_presenter.gd — Reactive UI Presenter

Marked @tool. Extends Control. Exposes game state as @export properties with property setters that call `_refresh_ui()` on change. `throw_controller` drives it exclusively via `_set_presenter_value(property_name, value)` which calls `node.set(property_name, value)`.

Property	Type	UI Effect
active_thrower	int (0/1)	Shows/hides CurrentThrowIcon; modulates row brightness
first_thrower	int (0/1)	Shows/hides FirstThrowIcon per row
player_on_nine	bool	Shows/hides player NineDartBadge with gold pulse
ai_on_nine	bool	Shows/hides AI NineDartBadge with gold pulse
checkout_route_visible	bool	Shows/hides CheckoutRoute container

Property	Type	UI Effect
checkout_route_thrower	int (0/1)	Reserved; unused at runtime

_refresh_motion() runs every _process tick. A 2.6 Hz sine wave drives: active-row brightness $\pm 6\%$, checkout route brightness $\pm 10\%$, nine-dart badge opacity $\pm 22\%$. Inactive rows are rendered at a fixed muted colour (0.74, 0.76, 0.86).

4.4 broadcast_backdrop.gd — Atmospheric Renderer

Marked @tool with a preview_motion_in_editor toggle. Extends Node2D. Draws a fully procedural broadcast arena in _draw() using configurable accent colors.

Layer	Technique	Animation
Arena gradient	44 horizontal rect bands tinted by accent colors	Static
LED ribbons	4 panels $\times$ 30 shimmer segments	fmod(phase*8, 72) scroll + per-segment sine shimmer
Stage truss	draw_line diagonals and angled struts	Static
Light beams	5 wide + 3 narrow polygons from ceiling	sin(phase*0.22) horizontal oscillation
Crowd silhouettes	74 rect+circle pairs along bottom row	sin(phase*0.12) wave motion
Haze blobs	6 large low-alpha circles	Lissajous drift (sin + cos at different frequencies)
Board spotlight	5 concentric circles	sin(phase*0.55) breathing radius and alpha
Camera split	Vertical bar at divider_x=900 with falloff and glow line	Static
Vignette	Dark rects on all 4 edges	Static

4.5 match_setup_modal.gd — Configuration UI

Shown at startup. Calls throw_controller.hold_for_setup() in _ready() to freeze the match. On 'Start Match' calls configure_match() with all selected values. Uses OptionButton.get_selected_id() / add_item(text, id) so option display text is decoupled from the integer values passed to game logic.

Option	Widget	Values
Starting score	OptionButton	301, 501
Match type	OptionButton	Legs (0), Sets (1)
Format length	OptionButton	Best of 5, 7, 9
Throws first	OptionButton	Player (0), AI (1), Random (2)
Entry rule	CheckBox	Double in (true) or Straight in (false)

4.6 stats_summary_modal.gd — Post-Leg/Match Report

Receives structured data from throw_controller._show_stats_summary() and dynamically builds two GridContainer tables by creating ColorRect + Label child pairs at runtime. Existing children are freed and rebuilt on each call to show_summary_data().

Table	Columns	Row highlight
SummaryTable (3 cols)	Stat name Player value AI value	Alternating dark/slightly-lighter
BreakdownTable (6 cols)	Leg Thrower Visit scores Darts Avg Checkout%	WIN_ROW_COLOR + gold thrower name for leg winner

The 'Close' / 'Next Set' button label is set dynamically. In between-set mode, clicking calls throw_controller.continue_after_summary() to reset leg counts and start the next set.

4.7 window_manager.gd — Display Manager

Extends Node. Listens for KEY_F11. Captures windowed size and position before entering fullscreen via DisplayServer; restores both on exit. Default windowed size is 1280×720 if no valid size can be read from the display server.

5. Data Flow

[illegible]

The `throw_controller` is the single orchestrator. All other nodes are either purely read-by-NodePath labels or receive push updates via method calls. There is no bidirectional dependency except the setup and summary modals, which each call one method on `throw_controller`.

6. Stat Tracking Schema

Statistics exist at three nested scopes, each with its own reset boundary:

Scope	Key variables	Reset point
Match	<code>_player_total_points</code> , <code>_player_darts_thrown</code> , <code>_player_first_nine_*</code> , <code>checkout totals</code> , <code>ton+ counts</code>	<code>reset_match()</code>
Leg	<code>_player_leg_darts</code> , <code>_player_leg_visits[]</code> , <code>_player_leg_first_nine_*</code> , <code>leg checkout/high-checkout</code>	<code>_reset_leg()</code>
Visit	<code>_visit_total</code> , <code>_visit_bust</code> , <code>_visit_checkout</code> , <code>_visit_started_on_checkout</code>	<code>_reset_visit_state()</code>

On leg completion, `_record_completed_leg()` serialises all leg-scope stats into a Dictionary and appends it to both `_match_leg_summaries` and `_current_set_leg_summaries`. The summary modal aggregates these via `_aggregate_summary_stats()`, which sums all additive fields and takes `max()` for high checkout.

Tracked statistics (per player, per scope)

- Points scored (all visited darts, 0 for bust visits)
- Darts thrown (total, and first-9 subset per leg)
- 3-dart average: $(\text{total points} \times 3) / \text{darts thrown}$
- First-9 average: same formula applied to first 9 darts of each leg
- Checkout attempts: visits where score was on a checkout-eligible number
- Checkout successes and percentage
- High checkout: highest score at start of a successful checkout visit
- 100+ / 140+ / 180 visit counts
- Nine-darter badge: live check that remaining darts ≤ 3 and route exists

7. Known Design Constraints & Coupling Points

Concern	Detail	Risk
Radius constants duplicated	dartboard.gd and throw_controller.gd each independently define the 7 ring radii.	High — mismatch silently misaligns visual rings with hit detection zones.
NodePath label coupling	throw_controller holds ~30 @export NodePath fields. get_node_or_null() returns null silently on broken paths.	Medium — renaming any scoreboard node in the scene breaks the corresponding label with no error.
String-based method calls	match_setup_modal and stats_summary_modal call throw_controller methods by string via call(). No static typing.	Low — typos cause silent no-ops at runtime; no compile-time check.
Sets mode: legs-per-set hardcoded	_legs_to_win_current_set() always returns 2 (best of 3). Not configurable via setup_modal.	Low — correct for standard darts rules but not flexible.
AI difficulty not configurable	control_skill and pressure affect player aim wobble only. AI wobble magnitude is hardcoded in _ai_angle_offset and _ai_radius_for_multiplier.	Low — no difficulty slider exposes AI accuracy to the player.
Checkout solver not cached	_checkout_route_for_score runs on every frame in _update_checkout_route(). $O(n^3)$ worst case.	Very Low — array sizes ($\sim 21 \times \sim 41 \times \sim 41$) make this negligible in practice.
Checkout tile positioning	_position_checkout_route() reassigns offset_top every _process tick; currently static but designed for future animation.	Very Low — minor redundant work each frame.

8. Extension Points

Feature	Approach
AI difficulty setting	Expose _ai_angle_offset magnitude and _ai_radius_for_multiplier wobble as @export floats. Add a difficulty OptionButton to match_setup_modal.
Second human player (local multiplayer)	Replace _begin_ai_visit / _throw_ai_dart with a second player's throw phase. The alternation logic in the phase machine requires no structural changes.
Sound effects	dart_landed and visit_complete signals are already emitted every dart/visit. Connect an audio node to these without touching game logic.
Persistent stats / leaderboard	_match_leg_summaries is a fully-populated Array[Dictionary] at match end. Serialise to JSON via JSON.stringify() and write with FileAccess.
Shared radius constants	Extract the 7 ring radii into a GDScript autoload singleton. Both dartboard.gd and throw_controller.gd import from it, eliminating duplication.
Animated checkout route	_position_checkout_route() already reads checkout_route_top as an @export. Tween this value to slide the route panel in/out on checkout detection.
Replay / dart history	The _results Array[Dictionary] persists for the duration of a visit and carries global_position. Extend _record_completed_leg to capture full dart-by-dart detail for playback.

9. Board Geometry Reference

Zone	Inner radius (px)	Outer radius (px)	Multiplier
Inner bull (double bull)	0	14	$\times 2$ (50 pts)

Zone	Inner radius (px)	Outer radius (px)	Multiplier
Outer bull (single bull)	14	35	×1 (25 pts)
Single (inner)	35	190	×1
Treble ring	190	222	×3
Single (outer)	222	318	×1
Double ring	318	350	×2
Outside / Miss	350+	—	0

Sector order (clockwise from top, matching BOARD_NUMBERS array):

20 1 18 4 13 6 10 15 2 17 3 19 7 16 8 11 14 9 12 5

START_ANGLE = $-\pi/2$ (top of board). SECTOR_ANGLE = $2\pi/20 = 18^\circ$. Segment boundary lines run from r=35 (outer bull edge) to r=350 (double outer).

10. Gameplay Flow Summary

```
[Launch]
→ MatchSetupModal visible; ThrowController.hold_for_setup() called
→ Player configures match and clicks 'Start Match'
→ configure_match() called; modal hidden; reset_match() runs

[Each Leg]
→ _reset_leg(): scores to starting_score; stats cleared
→ first thrower determined (alternates each leg)

[Player Visit]
→ LOCK_HEIGHT: aim line sweeps vertically
→ [click/Space]: y locked → LOCK_WIDTH
→ LOCK_WIDTH: aim line sweeps horizontally within chord
→ [click/Space]: dart lands → _land_player_dart()
→ LANDED delay → next dart or end of visit

[AI Visit]
→ AI_THROWING: darts fire at ai_dart_interval
→ each dart: _next_ai_target_label() → _local_position_for_target_label()
→ all darts thrown → ai_turn_end_delay → _finish_ai_visit()

[Leg end]
→ winner's score reaches 0 on a double
→ _complete_leg() → update counters
→ Legs mode: enough legs? → match complete; else next leg
→ Sets mode: enough legs in set? → _complete_set()
enough sets? → match complete
else → StatsSummaryModal (Next Set)

[Match Complete]
→ THROW_PHASE_MATCH_COMPLETE; StatsSummaryModal shown
→ Press R to reset
```